# REPORT

# FOR

# 2EC202CC23 – FPGA based System Design

**Semester – IV (Even)**

## Project Title:

# Matrix Multiplication Accelerator


**Report created by:**

**Parmar Jiya (24BEC096)**

**Jadeja Jaydeviba (24BEC077)**




**Nirma University**

**School of Technology, Institute of Technology**

**Electronics and Communication Engineering Department**

# Index

## 1. Abstract:

The multiplication of matrices is the most important and most utilized operation in digital signal processing, image processing, machine learning, and scientific computing. Matrix multiplications executed on traditional sequential processors typically consume considerable computing resources and time, particularly as the size of the matrices increases. To rectify this limitation, this project contains the design and implementation of a hardware-based matrix multiplication accelerator, using the Verilog hardware description language (HDL) on an FPGA platform.

This project contains a development of an N × N matrix multiplication accelerator, where the value of N is variable to allow flexibility in the size of matrices being used for execution. The architecture implements parallel multipliers in the form of Verilog generate blocks to permit the execution of multiple multiplication operations at the same time. The partial products are accumulated using the multiply-accumulate (MAC) technique to yield the final product matrix. This architecture for parallel hardware has substantially improved computational efficiency over that achievable through sequential implementations.

The matrices are stored internally, and the computed result matrix elements are sequentially displayed on a **7-segment display interface** implemented on the FPGA board. The design demonstrates efficient utilization of hardware resources while achieving faster computation through parallel processing.

This project demonstrates the benefits of using hardware accelerations with FPGA boards for performing heavy computations, such as matrix multiplication, providing a scalable solutions for many types of high performance digital applications.

## 2. Keywords:

Parameterized Hardware Design, Generate Block Parallel Multipliers, MAC Architecture, Matrix Storage using 2-D Arrays, State-Based Result Selection, Binary-to-Decimal

Conversion, 7-Segment Display Decoder

## 3. Literature survey/State of the art technology available:

FPGA technology has been applied to a number of different hardware accelerators designed specifically for accelerating matrix multiplication. One such accelerator is a systolic array-based architecture that was developed to support parallel processing and pipelining in order to improve computational speed. Another example is the use of tiled matrix multiplication accelerators based on FPGAs, which provide improved data reuse and memory bandwidth efficiency. It has been shown that hardware implementations significantly outshine traditional CPU-based methods when compared in terms of speed and energy efficiency.

## Some research papers:

**A]**

Title: *Design and Implementation of an FPGA-Based Tiled Matrix Multiplication Accelerator*

Authors: Zhaoqin Li, Sicheng Chen

**Main Idea:**

This research proposes the FPGA accelerator for matrix multiplication used in transformer neural networks. The design use tiling and parallel computation for increase data reuse and reduce memory access latency. Experimental result show significant speed improvement compared with CPU implementations.

**B]**

Title: *Design and FPGA Implementation of Systolic Array Architecture for Matrix Multiplication*

**Main Idea:**

This paper propose a systolic array architecture for performing matrix multiplication on FPGAs. The architecture combines parallel processing and pipelining, which improves the

computation speed compared to conventional method

**C]**

Title: *Optimization and Analysis of FPGA-Based Systolic Array for Matrix Multiplication*

**Main Idea:**

This work analyse the power consumption, area usage, and performance of FPGA matrix multiplication accelerators using an systolic arrays. The study demonstrates improved power efficiency and the optimized hardware resource utilizations.

## 4. Limitations or drawbacks of currently available technology:

Although hardware accelerators is significantly improve the speed of matrix multiplication, several limitations still exists in currently available technologies.

Limitations of current matrix multiplication accelerator technologies include :

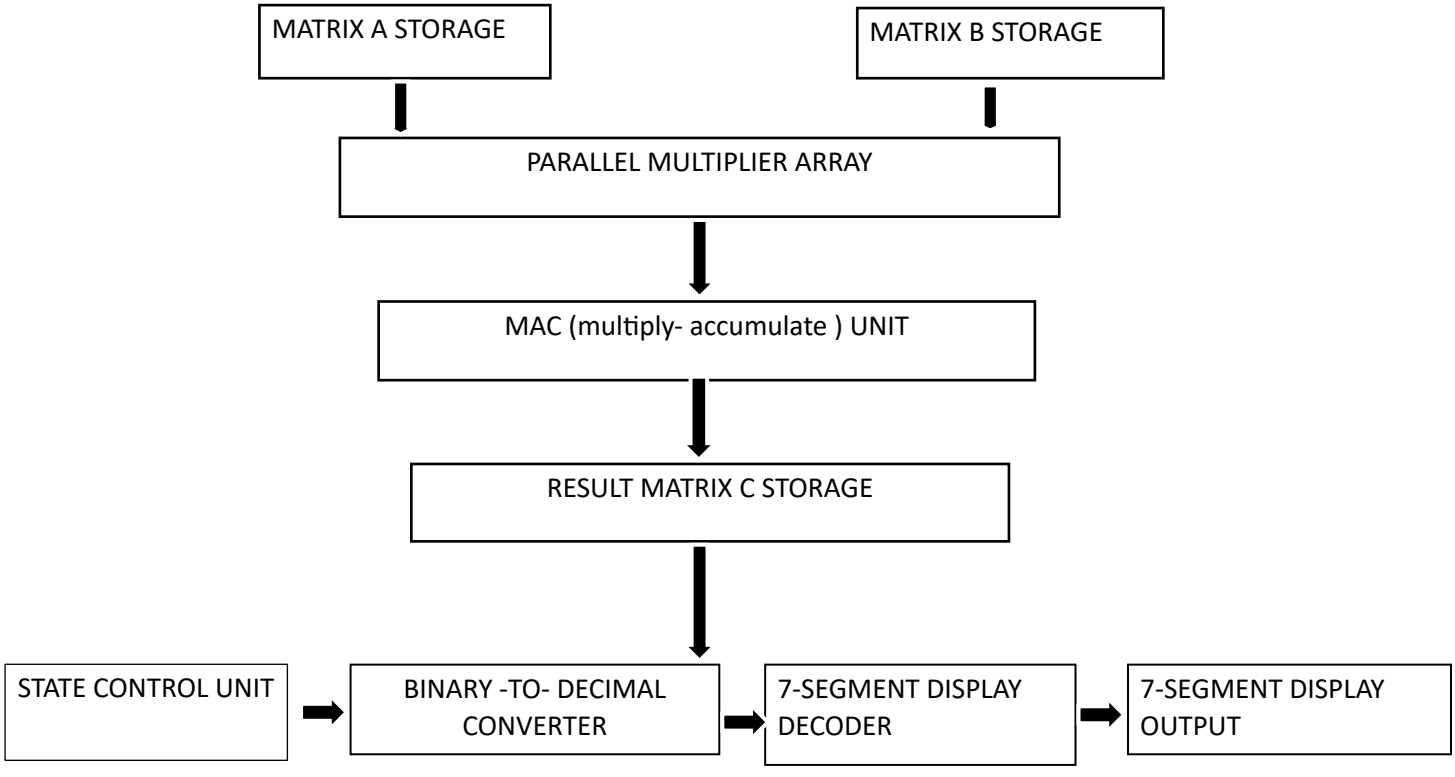
- limited FPGA hardware resources
- memory bandwidth constraints
- quantisation errors due to fixed-point arithmetic
- scalability issues for large matrices
- complex hardware design and debugging processes
- performance trade-offs between power consumption and computational speed

## 5. Proposed solution/methodology:

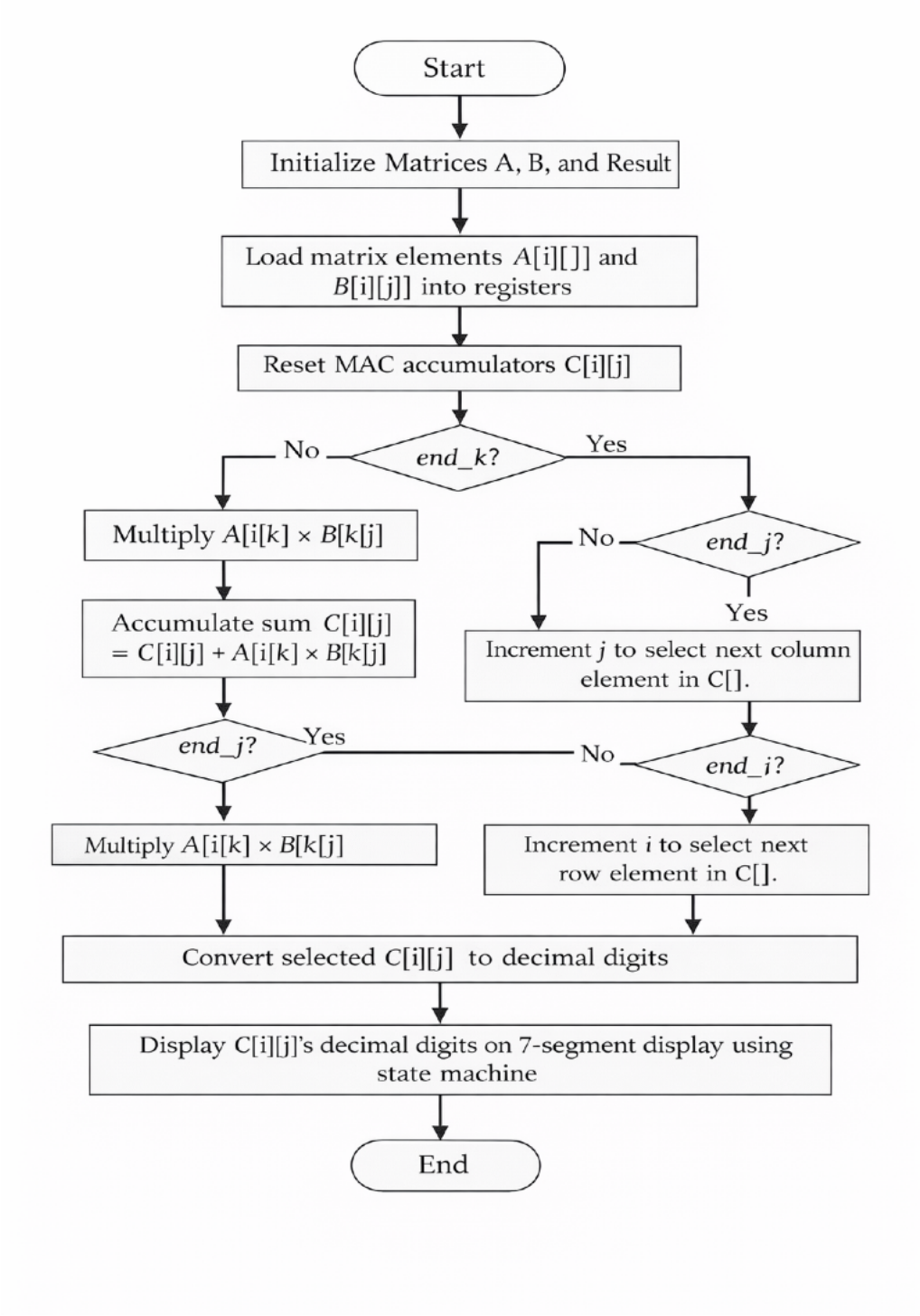
- To accelerate matrix multiplication, the hardware-based solution is proposed using Verilog HDL implemented on an FPGA platform. The proposed system performs parameterised N × N matrix multiplication, where the matrix size can be modified using design parameters. This makes the architecture flexible and scalable for different matrix dimensions.

- Both input matrices, A and B, are stored in two-dimensional register arrays within the device. Parallel multiplication occurs between respective elements of these two matrices with multiple parallel multipliers being created via Verilog generate blocks, which allows for multiple multiplications to go on at once. As a result, parallel computation of both input matrices greatly reduces the time needed to execute all of the multiplications compared to performing them sequentially in software.
- For each element of the result matrix **C**, a Multiply–Accumulate (MAC) operation is performed. The products of corresponding elements from matrices A and B are summed together to produce the final output value. The computation is synchronized with the clock signal to ensure correct sequential hardware operation.
- A state-based control mechanism will sequentially select from the result matrix after completing a multiplication operation. The selected value will then be converted from binary to decimal values via a series of divisions. After being converted to decimal form, the digit is sent to a display decoder (7-segment decoder) that decodes the digit into a format that can be displayed visually on the FPGA device.

## 6. Block diagram:

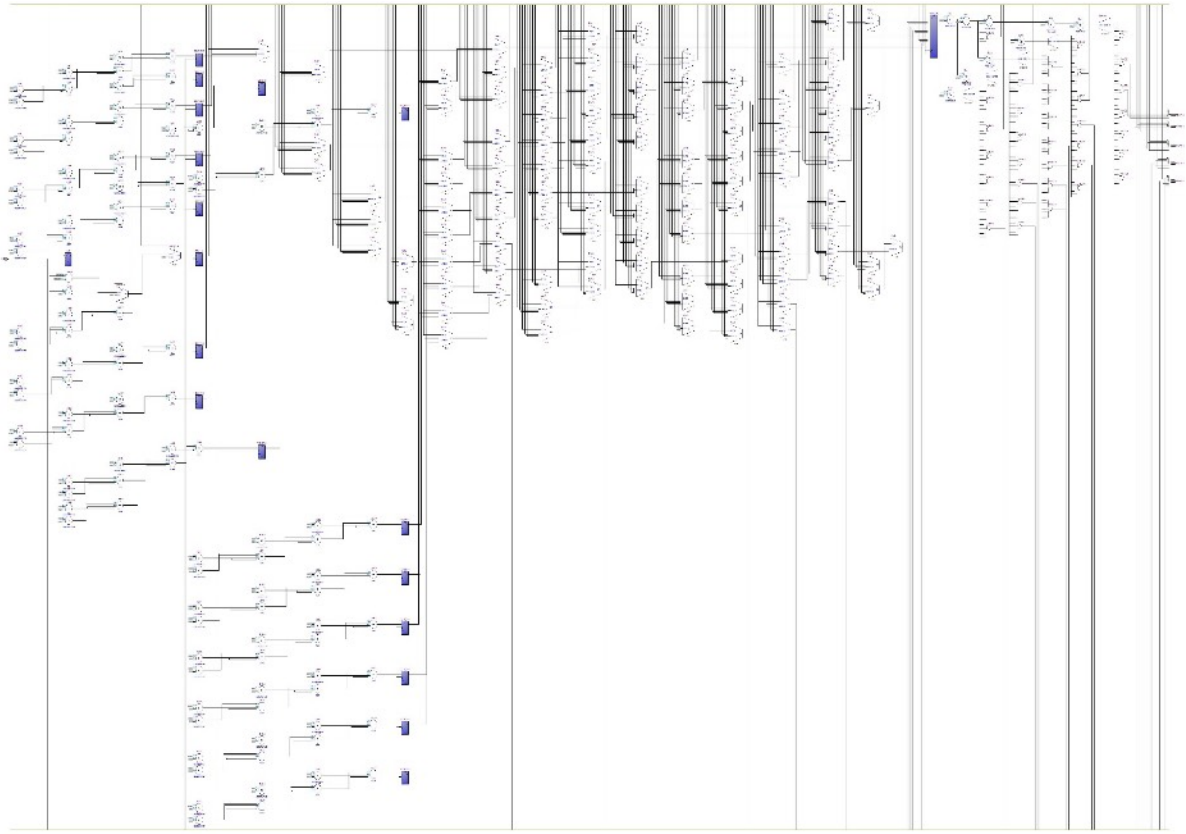


## 7. Flow chart of the code:

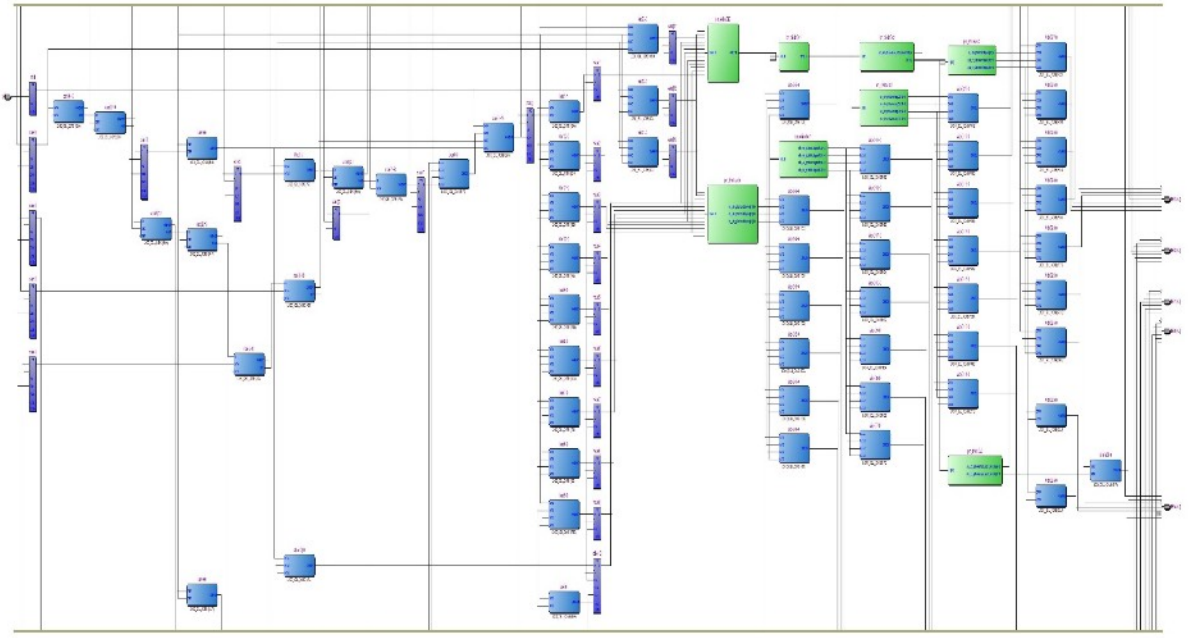


## 8. Results in terms of RTL, TTL, waveforms and Result tables:

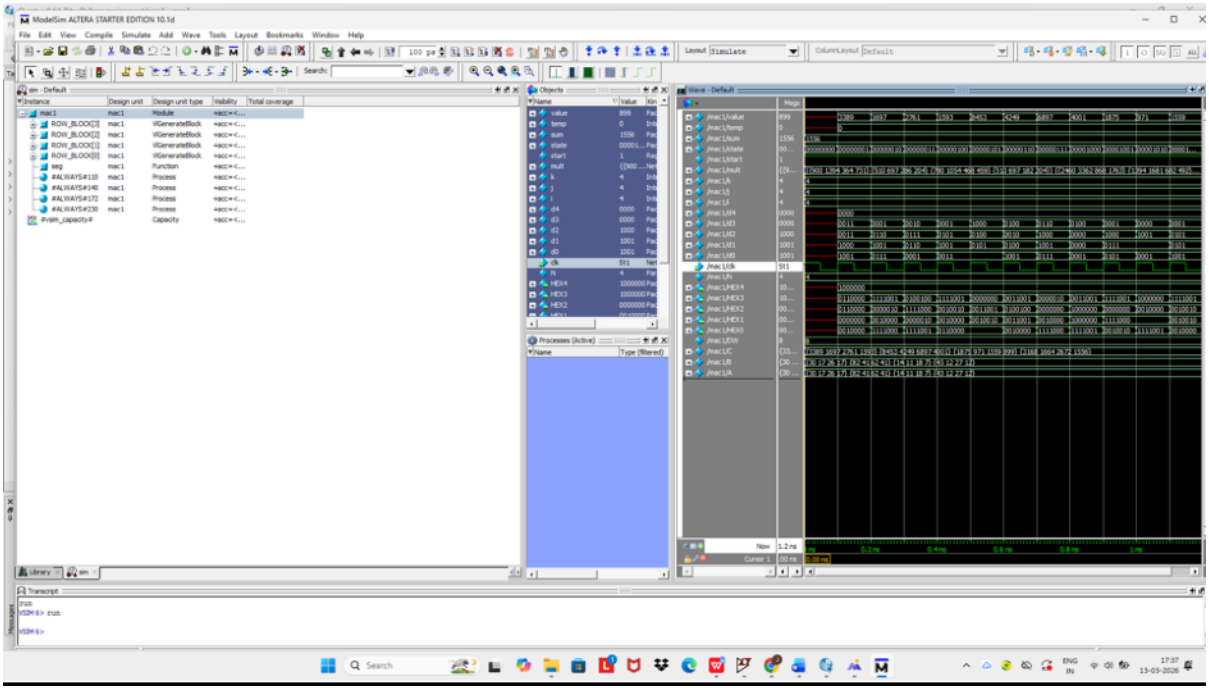
**RTL:**



## TTL:



## Simulation:



**RESULT TABLE:**

**Matrix entries:**

| | | | | | | |
|---|---|---|---|---|---|---|
| **A[0[0]** | **30** | **B[0][0]** | **30** | | **C[0][0]** | **3389** |
| **A[0][1]** | **17** | **B[0][1]** | **17** | | **C[0][1]** | **1697** |
| **A[0][2]** | **26** | **B[0][2]** | **26** | | **C[0][2]** | **2761** |
| **A[0][3]** | **17** | **B[0][3]** | **17** | | **C[0][3]** | **1593** |
| **A[1][0]** | **82** | **B[1][0]** | **82** | | **C[1][0]** | **8453** |
| **A[1][1]** | **41** | **B[1][1]** | **41** | | **C[1][1]** | **4249** |
| **A[1][2]** | **62** | **B[1][2]** | **62** | | **C[1][2]** | **6897** |
| **A[1][3]** | **41** | **B[1][3]** | **41** | | **C[1][3]** | **4001** |
| **A[2][0]** | **14** | **B[2][0]** | **14** | | **C[2][0]** | **1875** |
| **A[2][1]** | **11** | **B[2][1]** | **11** | | **C[2][1]** | **971** |
| **A[2][2]** | **18** | **B[2][2]** | **18** | | **C[2][2]** | **1559** |
| **A[2][3]** | **7** | **B[2][3]** | **7** | | **C[2][3]** | **899** |
| **A[3][0]** | **43** | **B[3][0]** | **43** | | **C[3][0]** | **3168** |
| **A[3][1]** | **12** | **B[3][1]** | **12** | | **C[3][1]** | **1664** |
| **A[3][2]** | **27** | **B[3][2]** | **27** | | **C[3][2]** | **2672** |
| **A[3][3]** | **12** | **B[3][3]** | **12** | | **C[3][3]** | **1556** |

## 9. Conclusion:

In this project, a hardware-based N × N matrix multiplication accelerator was successfully designed and implemented using Verilog HDL on an FPGA platform. The system utilizes a parameterized architecture, allowing the matrix dimension to be easily modified without changing the overall design structure. This flexibility makes the design scalable for different matrix sizes.

The use of parallel multipliers created using Verilog generate constructs enables the

accelerator to perform multiple multiplication operations simultaneously. To efficiently compute each element of the result matrix, the partial products generated by the multipliers are added together through a Multiply-Accumulate (MAC) mechanism. The parallel hardware implementation of the accelerators provides a significant increase in computational speed compared to traditional software-based, sequential methods.

The final computed results are stored in a result matrix register array, and then selected one by one using a state-based control mechanism. The selected result matrix elements are converted from binary format to decimal digits and displayed on a 7-segment display interface allowing for real-time visualization of the computed values on the FPGA board.

In conclusion, the proposed design supports the argument that accelerators based on FPGA (first-in-first-out) provide speed and effective use of resources when performing computationally intensive operations such as matrix multiplication. The project demonstrates the benefits of parallel processing in digital hardware systems and lays the groundwork for creating more complex matrix computation accelerators for applications like signal processing, scientific computing, and machine learning.

## 10.References:


1. Palnitkar, S. Verilog HDL: A Guide to Digital Design and Synthesis. Prentice Hall
2. Mano, M. M. Digital Design. Pearson Education.
3. Chu, P. P. FPGA Prototyping by Verilog Examples. Wiley.
4. Intel Corporation. Quartus Prime FPGA Design Software Documentation.
5. Siemens EDA. ModelSim HDL Simulator User Manual.
6. Patterson, D. A., Hennessy, J. L. Computer Organization and Design. Morgan Kaufmann.
7. Floyd, T. L. Digital Fundamentals. Pearson.

## 11. Code in the Appendix:

MATRIX MULTIPLICATION ACCELERATOR WITH DISPLAY:

```
module mac1
#(
   parameter N  = 4,
   parameter DW = 8
)
(    input clk,
    output reg [6:0] HEX0,
    output reg [6:0] HEX1,
    output reg [6:0] HEX2,
    output reg [6:0] HEX3,
   output reg [6:0] HEX4
);
```

MATRIX STORAGE:

```
reg signed [DW-1:0] A [0:N-1][0:N-1];
reg signed [DW-1:0] B [0:N-1][0:N-1];
reg signed [2*DW+4:0] C [0:N-1][0:N-1];

integer i,j,k;
```

MATRIX INITIALIZATION:

```
initial
begin
A[0][0] = 30;
A[0][1] = 17;
A[0][2] = 26;
A[0][3] = 17;
A[1][0] = 82;
```

```
A[1][1] = 41;
A[1][2] = 62;
A[1][3] = 41;
A[2][0] = 14;
A[2][1] = 11;
A[2][2] = 18;
A[2][3] = 7;
A[3][0] = 43;
A[3][1] = 12;
A[3][2] = 27;
A[3][3] = 12;

B[0][0] = 30;
B[0][1] = 17;
B[0][2] = 26;
B[0][3] = 17;
B[1][0] = 82;
B[1][1] = 41;
B[1][2] = 62;
B[1][3] = 41;
B[2][0] = 14;
B[2][1] = 11;
B[2][2] = 18;
B[2][3] = 7;
B[3][0] = 43;
B[3][1] = 12;
B[3][2] = 27;
B[3][3] = 12;
end
```

PARALLEL MULTIPLIERS:

```
    wire signed [2*DW-1:0] mult [0:N-1][0:N-1][0:N-1];
    genvar r,c,t;
generate
    for(r=0; r<N; r=r+1)
    begin: ROW_BLOCK
        for(c=0; c<N; c=c+1)
        begin: COL_BLOCK
            for(t=0; t<N; t=t+1)
            begin: MULT_BLOCK
                assign mult[r][c][t] = A[r][t] * B[t][c];
            end
        end
    end
    endgenerate
```

MAC ACCUMULATION:

```
    reg signed [2*DW+4:0] sum;
    always @(posedge clk)
    begin
    for(i=0;i<N;i=i+1)
    begin
        for(j=0;j<N;j=j+1)
        begin
            sum = 0;
            for(k=0;k<N;k=k+1)
                sum = sum + mult[i][j][k];
            C[i][j] <= sum;
        end
    end
    end
```

RESULT SELECTION:

```
reg [7:0] state = 0;
reg start = 0;
reg signed [2*DW+4:0] value;
always @(posedge clk)
begin
if(start == 0)
begin
   start <= 1;   // wait one cycle for C matrix to compute
   state <= 0;
end
else
begin
   value <= C[state / N][state % N];
   if(state == (N*N - 1))
      state <= 0;
   else
      state <= state + 1;
end
end
```

BINARY TO DECIMAL DIGITS:

```
integer temp;
reg [3:0] d0,d1,d2,d3,d4;
always @(*)
begin
temp = value;
d0 = temp % 10;
temp = temp / 10;
d1 = temp % 10;
```

```
temp = temp / 10;
d2 = temp % 10;
temp = temp / 10;
d3 = temp % 10;
temp = temp / 10;
d4 = temp % 10;
end
```

7 SEGMENT DECODER:

```
function [6:0] seg;
input [3:0] digit;

begin
case(digit)

0: seg = 7'b1000000;
1: seg = 7'b1111001;
2: seg = 7'b0100100;
3: seg = 7'b0110000;
4: seg = 7'b0011001;
5: seg = 7'b0010010;
6: seg = 7'b0000010;
7: seg = 7'b1111000;
8: seg = 7'b0000000;
9: seg = 7'b0010000;
default: seg = 7'b1111111;
endcase
end
endfunction
```

DISPLAY OUTPUT:

```
always @(*)
begin

HEX0 = seg(d0);
HEX1 = seg(d1);
HEX2 = seg(d2);
HEX3 = seg(d3);
HEX4 = seg(d4);
end
endmodule
```